

# React Native (Expo) Testing Demo — Jest + @testing-library/react-native

---

This is a small **Expo-based React Native** sample app designed for demos of:

- **Unit tests** (pure functions + components)
  - **Integration tests** (App combines multiple parts)
  - **Smoke tests** (renders without crashing)
  - **Sanity tests** (a key feature works right now)
  - **Regression tests** (features still work after changes)
  - **HTML test report output**
- 

## Download / Get the project

Option A: Download as a ZIP (GitHub)

- Download the repo ZIP
- Extract it
- Open the extracted folder in VS Code / Cursor

Option B: Clone with Git

```
git clone https://github.com/neonbranch/CPSC-357-Lab.git
cd lab8
```

If you already opened this project folder in your editor/terminal, you can skip the `cd`.

---

## Notes

- **Unit test**
  - **Definition:** tests one small piece in isolation (a function or a single component).
  - **Use it for:** pure logic, small UI behavior.
  - **In this repo:** `__tests__/math.test.ts`, `__tests__/Counter.test.tsx`
- **Integration test**
  - **Definition:** tests multiple parts working together (screen + components + services).
  - **Use it for:** verifying a full screen/feature still works end-to-end in code.
  - **In this repo:** `__tests__/App.test.tsx` (App + greeting + math + Counter)
- **Smoke test**
  - **Definition:** "does it render without crashing?"
  - **Use it for:** quick confidence after refactors.

- **In this repo:** App render check in `__tests__/App.test.tsx`
- **Sanity test**
  - **Definition:** quick check that the most important feature still works right now.
  - **Use it for:** after a UI/logic change (fast confirmation).
  - **In this repo:** press Increment once → counter becomes `1` in `__tests__/App.test.tsx`
- **Regression test**
  - **Definition:** protects existing behavior from breaking in the future.
  - **Use it for:** features that must not change (or bugs you already fixed once).
  - **In this repo:** increment/increment/decrement sequence stays correct in `__tests__/App.test.tsx`

## Requirements

- Node.js **18+**
- npm (comes with Node)
- Expo Go app on your phone (optional, for running on device)

---

## Install dependencies

From the project root:

```
npm install
```

---

## Run the app (Expo)

Start Metro / Expo Dev Server:

```
npx expo start
```

Then:

- Press `a` to open Android emulator (if installed), or
- Scan the QR code in Expo Go on your phone

---

## Project structure

```
lab8/  
  App.tsx  
  app.json  
  babel.config.js
```

```
jest.config.js
jest.d.ts
jest.setup.js
package.json
tsconfig.json

src/
  components/
    Counter.tsx
  services/
    greeting.ts
  utils/
    math.ts

__tests__/
  App.test.tsx
  Counter.test.tsx
  math.test.ts

test-reports/
  test-report.html  <-- created by Jest (HTML report)

coverage/           <-- created by Jest coverage
```

---

## What the app does

- **Counter:** increment/decrement buttons
- **Utility:** `add(a, b)` demonstrates pure function unit tests
- **Service:** `getGreeting(name)` demonstrates a simple service + mocking in tests
- **App:** shows greeting + math result + counter

---

## Jest setup in this project

### Where Jest is configured

- `jest.config.js`
  - Uses the **Expo preset:** `preset: 'jest-expo'`
  - Loads test matchers via `setupFilesAfterEnv`
  - Writes an **HTML report** to `test-reports/test-report.html`
  - Enables coverage output (text + lcov + html)
- `jest.setup.js`
  - Loads extra matchers like `toHaveTextContent`:

```
import '@testing-library/jest-native/extend-expect';
```

## Key `package.json` scripts

Open `package.json` and look for:

- `npm test`
  - Runs all tests
- `npm run test:coverage`
  - Runs tests + prints coverage table + writes `coverage/`
- `npm run test:html`
  - Runs tests + writes HTML report to `test-reports/test-report.html`

List scripts:

```
npm run
```

---

## Testing Library basics used here

This project uses **@testing-library/react-native**. The “pattern” is:

1. **Render** a component:

```
const { getByTestId } = render(<App />);
```

2. **Find UI elements** using `testID` (recommended for beginners):

```
const incrementButton = getByTestId('increment-button');
```

3. **Fire events** to simulate user actions:

```
fireEvent.press(incrementButton);
```

4. **Assert expected UI output:**

```
expect(getByTestId('counter-value')).toHaveTextContent('1');
```

## Why we use `testID`

Using `testID` makes tests:

- easy to read
- resilient to text/label changes

- consistent across iOS/Android
- 

## Types of tests in this repo (with examples)

### A) Unit tests

- **Utility unit test:** `__tests__/math.test.ts`
  - Tests `add(a, b)` with simple inputs/outputs
- **Component unit test:** `__tests__/Counter.test.tsx`
  - Renders `<Counter />`
  - Presses increment/decrement buttons
  - Asserts the displayed count

### B) Integration test

- `__tests__/App.test.tsx`
  - Renders `<App />` (which includes greeting + math + Counter)
  - Verifies the combined UI works together

### C) Smoke test

- `__tests__/App.test.tsx`
  - "renders without crashing" style check

### D) Sanity test

- `__tests__/App.test.tsx`
  - A quick "increment works" test to confirm key behavior after changes

### E) Regression test

- `__tests__/App.test.tsx`
  - Confirms increment/decrement still work correctly (after updates)

### Mocking example (`jest.fn`)

In `__tests__/App.test.tsx`:

- We mock `getGreeting()` using `jest.spyOn(...)`
- We return a fake greeting string
- We assert the App shows the mocked value

This demonstrates how to test a component **without relying on the real service implementation**.

---

## Step-by-step: run tests from `__tests__`

### 1) Run all tests

```
npm test
```

### 1a) What you should see in the console

- PASS \_\_tests\_\_/... for each test file
- A summary like:
  - Test Suites: X passed, X total
  - Tests: Y passed, Y total

### Quick commands (most used)

- Run all tests:

```
npm test
```

- Run coverage:

```
npm run test:coverage
```

- Generate HTML test report:

```
npm run test:html
```

### Jest automatically finds tests in:

- \_\_tests\_\_/\*\*/\*.test.ts
- \_\_tests\_\_/\*\*/\*.test.tsx

### 2) Run ONE specific test file

```
npx jest __tests__/App.test.tsx
```

Or:

```
npx jest __tests__/Counter.test.tsx
```

### 3) Watch mode (optional)

```
npx jest --watch
```

---

## Coverage reports

Run:

```
npm run test:coverage
```

Outputs:

- Coverage table in the console
  - `coverage/` folder (including HTML coverage report)
- 

## HTML test report

Run:

```
npm run test:html
```

Output file:

- `test-reports/test-report.html`

### View the HTML report

1. Open the folder `test-reports/`
2. Open `test-report.html` (double-click or open with a browser)

Tip: In VS Code/Cursor you can right-click `test-report.html` → "Reveal in File Explorer".

---

## Today's outline

### Class tasks

1. Run the app
  - `npx expo start`
2. Confirm Jest is included in this project (team check)
  - Open `package.json` and find the scripts:
    - `test, test:coverage, test:html`
  - Open `jest.config.js` and confirm:
    - `preset: 'jest-expo'`

- HTML reporter writes to `test-reports/test-report.html`
  - Open `jest.setup.js` and confirm matchers are loaded.
3. Run the existing tests (baseline)
- `npm test`
4. As a team, pick one component to test
- Suggested: `src/components/Counter.tsx`
5. Test that component (write or extend one test)
- Open the matching test file in `__tests__`/ (example: `__tests__/Counter.test.tsx`)
  - Add one small test using:
    - `render(...)`
    - `getByTestId(...)`
    - `fireEvent.press(...)`
    - `expect(...)`
6. Generate reports
- Console coverage: `npm run test:coverage`
  - HTML report: `npm run test:html` → open `test-reports/test-report.html`